discriminative power in the fitted logistic regression models. For each Cursor-adopting repository, we perform 1:3 nearest-neighbor matching (three controls per treated unit). While 1:1 or 1:2 is most common [25, 96], many adopters matched the same control during 1:1 matching, so 1:3 provides higher control group diversity. We additionally match only repositories with the same primary language (queried from GitHub API, unavailable in GHArchive), controlling for language-specific LLM performance differences [35, 107]. This yields a matched "never-treated" group with similar propensity score distributions and pre-adoption characteristics (see Appendix). Following standard quasi-experimental terminology, we refer to the sample of repositories adopting Cursor as the *treatment group* and the matched "never-treated" repositories not using Cursor as the *control group* in the remainder of this paper.

## 3.2 Metrics

For each repository in the treatment and control group, we collect monthly outcome metrics and time-varying covariates from January 2024 to August 2025. This ensures that there are at least six months of observations pre- and post-adoption for the treatment group and abundant comparison observations from the control group in each month, providing statistical power for DiD-based causal inference. Note that the dataset is an *unbalanced panel*, as not all repositories have observations over the entire observation period.

*3.2.1 Outcomes.* For repository $i$ at month $t$, we collect two outcome metrics related to *development velocity*, a key dimension of software engineering productivity [38, 85]:

- Commits$_{it}$: Number of commits in repository $i$ at month $t$;
- Lines Added$_{it}$: Total lines added, summed over all commits in repository $i$ at month $t$.

Both have been used as productivity proxies [74, 83, 100] with moderate-to-strong correlation with perceived productivity [89].

Software quality is multi-faceted and difficult to capture with a single metric [36, 51, 71, 90]. Quality can be pivoted on defect density [55], specification rigor [50], user satisfaction [45], or technical debt [47]. However, many metrics cannot be reliably and scalably collected from version control data. In this study, we take the technical debt perspective [80] and test three source code maintainability metrics that can be reasonably estimated with static analysis: *static analysis warnings*, *duplicate line density*, and *code complexity*. All three are arguably positively correlated with project-level technical debt and negatively correlated with perceived code quality. Specifically, we use a local SonarQube Community server [16] to compute these outcome metrics for repository $i$ at month $t$:

- Static Analysis Warnings$_{it}$: Total number of reliability, maintainability, and security issues for repository $i$ at month $t$, as detected by SonarQube's static analysis. We refer to them as warnings, since static analysis can generate false positives [56]; this metric should be viewed as an estimate of the effort required to review potential issues in a project.
- Duplicate Line Density$_{it}$: Percentage of duplicated lines in codebase for repository $i$ at month $t$. SonarQube's definition varies across programming languages, but usually requires at least 10 consecutive duplicate statements or 100 duplicate tokens to mark a block as duplicate [18].
- Code Complexity$_{it}$: Overall cognitive complexity [32] of codebase for repository $i$ at month $t$. Per SonarQube [32], this metric quantifies code understandability and aligns better with modern coding practices than classic cyclomatic complexity [82].

*3.2.2 Time-Varying Covariates.* We control for the following time-varying covariates in our models for all treatment and control repositories over the entire observation period (Jan 2024 to Aug 2025): lines of code, age (days), number of contributors at month $t$, number of stars received at month $t$, number of issues opened at month $t$, and number of issue comments added at month $t$. Lines of code is collected from SonarQube [16] along with outcome metrics; number of contributors is estimated from version control history; remaining covariates are estimated from GHArchive event data [1]. Multi-collinearity analysis reveals that number of issues opened and number of issue comments added are highly correlated (Pearson's $\rho > 0.7$), so we exclude issue comments from subsequent modeling.

## 3.3 Difference-in-Differences

*3.3.1 Background.* DiD is an established econometric technique for causal inference in observational data [33, 41], with growing adoption in software engineering [34, 49, 86]. The key idea is to compare outcome changes in a treatment group to those in a control group (i.e., those not-yet-treated or never-treated) over the same observation periods. The name "difference-in-differences" originates from the fact that temporal changes are first differenced before differencing the outcome changes between the two groups, effectively isolating the effect of an intervention from other factors that affect all repositories similarly over the same period.

A DiD design critically relies on the *parallel trends assumption* for a causal interpretation: Absent treatment, the treatment and control group should, on average, follow similar outcome trajectories over the same period. While this assumption is generally not directly testable (would need a time machine), *pre-trend tests* are often used for assessing the *plausibility* of this assumption: If the model predicts similar outcomes for the treatment and control groups *before the adoption*, it is more plausible that the treatment group would evolve similarly if they were not exposed to the treatment. Apart from pre-trend tests, a matching process that controls for observable differences pre-adoption (e.g., Section 3.1.3) also strengthens the plausibility of this assumption in a specific research context.

In a DiD design, a *staggered adoption* setting comes with both promises and perils [23, 29]. In this setting, treatments occur at different times across cohorts rather than simultaneously (as in our case, Figure 2), and each treatment unit has repeated observations both before and after treatment. This setting enables repeated, multiple natural experiments: Later-adopters in the treatment group can serve as additional controls for those adopting before them, since they remain untreated during earlier periods. However, a staggered adoption setting also presents significant mathematical challenges to achieve consistent, efficient, and unbiased estimation of the causal effect [23, 44, 54] with ongoing active research [29, 31].

*3.3.2 The Estimation Targets.* In a DiD design, researchers are often interested in estimating the following two types of causal effects: (1) $ATT$, the average treatment effect on treated, and (2) $ATT_h$, the "horizon-average" treatment effect in a specific horizon $h$ (i.e., the